Introduction to Data Science — Assignment 4

Instructors: **Dr. Razavi, Dr. Yaghoobzadeh**

TAs: Behrad Binaei Haghighi, Mehrad Liviyan, Maryam Vali

Deadline: 23th Khordad

Introduction

This fourth assignment of the “Data Science” course at the University of Tehran is designed to equip students with practical deep learning skills across three distinct domains. In the first project, students will train a Multi-Layer Perceptron (MLP) on the California Housing dataset, focusing on feature engineering and tabular data workflows. The second project requires building a Convolutional Neural Network (CNN) for image classification, emphasizing mathematical foundations, robustness analysis, and model explainability via Grad-CAM. Finally, the third project utilizes Recurrent Neural Networks (RNNs) for Knowledge Tracing on educational time-series data to predict student performance. Collectively,

Task 1 : MLP

In this task, you are supposed to work with the California Housing dataset. This dataset contains demographic, socioeconomic, and geographical information collected from different districts in California. Your goal is to train a Multi Layer Perceptron (MLP) neural network to predict median house prices based on these features.

Throughout this task, you will perform a complete Deep Learning workflow including data exploration, preprocessing, model training, feature engineering, evaluation, and error analysis.

1- Dataset Loading

This dataset consists of several numerical attributes describing different districts in California.

Some important features include:

1. MedInc: Median income in the district.
2. HouseAge: Median house age.
3. AveRooms: Average number of rooms per household.
4. AveBedrms: Average number of bedrooms per household.
5. Population: Total population of the district.
6. AveOccup: Average household occupancy.
7. Latitude: Geographical latitude.
8. Longitude: Geographical longitude.

The target variable is:

- MedHouseVal: Median house value.

Now let's dive into the steps of this task one by one.

2- Exploratory Data Analysis (EDA)

Before building any models, carefully investigate the dataset. Begin by examining basic properties such as the number of samples and features, data types, missing values, and statistical summaries.

Next, analyze the distribution of the features and the target variable using visualizations like histograms, box plots, scatter plots, and correlation heatmaps. Your analysis should investigate skewed distributions, identify potential outliers, highlight strongly correlated features, and explore relationships between the input features and house prices.

Finally, summarize your observations and discuss how these characteristics—such as the need for normalization or the potential for feature engineering—might affect model performance.

3- Data Preparation

First, load the dataset into a pandas DataFrame and separate the input features (X) from the target variable (y). Split the dataset into training, validation, and test sets using a standard ratio, such as 70% for training, 15% for validation, and 15% for testing.

After splitting, normalize the numerical features using a scaler like StandardScaler. To prevent data leakage, ensure that the scaler is fitted exclusively on the training set before transforming the validation and test sets.

Finally, convert the processed data into tensors suitable for your chosen deep learning framework.

4- Baseline MLP Model

In this section, you will build a baseline Multi Layer Perceptron using only the original dataset features. Your model should inherit from `torch.nn.Module` class

The specific architecture is left to your discretion and may include multiple hidden layers, various numbers of neurons, and different activation functions. The primary purpose of this initial model is to establish a performance baseline for subsequent comparisons.

5- Model Training

Train your baseline model using a standard regression loss function, such as Mean Squared Error (MSE), and an appropriate optimizer like Adam or SGD.

During the training process, monitor both the training and validation losses. You are highly encouraged to implement Early Stopping and Model Checkpointing to prevent overfitting and to preserve the best-performing model weights.

Experiment with hyperparameters—including learning rate, batch size, number of epochs, and network architecture—and observe how these adjustments impact overall performance.

6- Feature Engineering

While deep learning models can automatically learn complex patterns, manual feature engineering remains highly beneficial for tabular datasets. Create additional, meaningful features based on your understanding of the problem domain.

Examples of potential features include rooms per household, bedrooms per room, population density, income-based ratios, or log-transformed variables. Propose and evaluate your own engineered features, analyzing whether they provide valuable information for predicting house prices.

7- Enhanced MLP Model

Train a second MLP model incorporating your newly engineered features. The objective here is to determine whether feature engineering improves generalization, prediction accuracy, and training stability. Compare the training behavior and convergence of this enhanced model against your initial baseline.

8- Model Evaluation and Comparison

Evaluate the performance of both models on the test set using standard regression metrics, including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the R^2 Score. Compare the models in terms of prediction accuracy, validation stability, convergence speed, and susceptibility to overfitting. Discuss whether the engineered features improved the model's performance and provide reasonable explanations for the observed results.

9- Error Analysis

In this section, perform a detailed investigation of the prediction errors made by your best-performing model.

Compare the predicted values against the actual values and plot the error distributions. Identify challenging samples, investigate potential outliers, and analyze specific scenarios where the model performs poorly. Suggest plausible explanations for these inaccuracies and discuss what additional features or preprocessing techniques could mitigate them.

10- Conclusion and Future Work

Summarize the complete workflow and your core findings. Discuss the impact of data preprocessing, the effectiveness of feature engineering, and the general strengths and limitations of applying MLPs to tabular data.

Finally, answer the following question:

- *Can feature engineering still improve the performance of deep learning models on structured tabular datasets?*

Support your answer with the empirical results obtained during this project.

Finally, suggest potential future improvements, such as advanced hyperparameter tuning, batch normalization, alternative regularization methods, dimensionality reduction, ensemble approaches, or different neural network architectures.

Task 2 : CNN

Introduction

In this project, you will work with the Fruits and Vegetables Image Recognition dataset from Kaggle, which contains high-quality RGB images categorized into predefined splits. Beyond achieving high classification accuracy, this task emphasizes understanding the mathematical foundations of Convolutional Neural Networks (CNNs), building robust preprocessing pipelines without data leakage, visualizing model decisions using Grad-CAM, and conducting quantitative robustness analyses to determine whether the model learns actual morphological "shapes" or merely memorizes "color."

1- Dataset Description & Configuration

You must select exactly 10 classes from the dataset strategically: choose some classes with high color similarity to challenge the model's discriminative power, alongside others with distinct morphological differences (a suggested baseline includes Apple, Banana, Bell Pepper, Carrot, Grapes, Kiwi, Lemon, Orange, Pear, and Tomato). If you deviate from this baseline, you must provide a scientific justification.

Resize all images to 128*128 pixels, standardize to RGB, and strictly use the existing Train/Validation/Test splits. In your report, detail the exact sample count per split for each class and plot a bar chart to analyze class distribution. Is there a class imbalance? If so, how does it affect the choice of evaluation metric?

2- Statistical EDA and Preprocessing

2.1 Color Distribution Analysis: For each class, calculate the mean values of the R, G, and B channels. Visualize this data by plotting a grouped bar chart (showing R/G/B bars side-by-side) and a pairwise RGB scatter plot (R vs G, R vs B, G vs B) to identify class clusters. Additionally, compute the Euclidean distance matrix in RGB space between all class pairs and display it as a heatmap.

- **Analytical Question 1:** Based on the color distance matrix, which 3 class pairs have the highest risk of confusion? Compare this prediction at the end with the model's actual Confusion Matrix and evaluate the accuracy of your EDA analysis.

2.2 Global Normalization: Calculate the mean (μ) and standard deviation (σ) for each color channel exclusively across the Training Set, and explicitly report these values. Apply Z-score normalization to the Train, Validation, and Test sets using these training statistics. Explain analytically why computing normalization statistics on the Test Set would constitute Data Leakage.

2.3 Data Augmentation Pipeline: Design a training-only augmentation pipeline incorporating random rotation (up to 15 degrees), random brightness/contrast jitter, random horizontal flips, and optionally, random cutout to improve robustness. Display a grid of at least 5 original images alongside their augmented versions to visually demonstrate these transformations.

3- Mathematical Foundations

Before any implementation, a mathematical understanding of the architecture is mandatory. Calculations in this section must be presented in the PDF report **step-by-step with formulas**.

Consider the following hypothetical architecture with input **128 × 128 × 3**:

Layer	Type	Parameters
1st	Convolution	32 Filters | Kernel 5×5 | Stride 1 | Padding 2
2nd	Max-Pooling	Window 2×2 | Stride 2
3rd	Convolution	64 Filters | Kernel 3×3 | Stride 1 | Padding 1

Question 1: Calculate the output dimensions (H × W × C) at each of the three stages using the following formula:

$$Output\ Size\ =\ [(Input\ +\ 2P\ -\ K)/S]\ +\ 1$$

Question 2: Calculate the exact number of trainable parameters (Weights + Biases) for each of the three layers. Also write out the calculation formula.

Question 3: Using the following recursive formula, compute the Receptive Field of a neuron at the output of the third layer relative to the original input image:

$$RF_l = RF_{l-1} + (K_l - 1) \times \prod S_i$$

Question 4: If we change the Stride of the first layer from 1 to 2, what quantitative change will occur in the Receptive Field of the third layer? What trade-off does this change create between spatial information and computational efficiency?

4- Model Architecture and Training

Implement a modular model using PyTorch or TensorFlow/Keras. Design a Feature Extractor with at least four convolutional blocks following the pattern:

[Conv2D → Batch Normalization → ReLU → MaxPool2D]

Begin with 32 filters and double the count in each subsequent block to facilitate hierarchical feature learning. For the Classifier Head, utilize Global Average Pooling (GAP) instead of a Flatten layer, and explain why GAP is advantageous regarding parameter reduction and overfitting.

Training Configuration

Component	Value / Setting
Optimizer	Adam (lr=1e-3)
Loss Function	Cross-Entropy Loss
LR Scheduler	ReduceLROnPlateau (patience=3, factor=0.5)
Batch Size	32 or 64
Max Epochs	Up to 50 (with Early Stopping)
Early Stopping	Based on Validation Loss (patience=5)

Training Documentation Requirements: Document your training process by plotting Training/Validation Loss and Accuracy on shared axes (versus epochs), explicitly marking the Early Stopping point. Report the average training time per epoch. Answer analytically: Is there visual evidence of overfitting in the curves? In which epochs did the LR scheduler activate?

5- Robustness Analysis: Color vs. Shape

This critical analysis determines if your model learned geometric shapes or merely memorized colors. Conduct the following tests and populate the comparison table below:

- **Test A (Baseline):** Report overall Test Set accuracy, plot the normalized Confusion Matrix, and generate a Classification Report (Precision, Recall, F1).
- **Test B (Grayscale Challenge):** Convert Test images to Grayscale (replicated across 3 channels) and recalculate accuracy and the confusion matrix.
- **Test C (Color Channel Swap):** Swap the R and B channels of the Test images (e.g., turning bananas purple) and evaluate the performance.
- **Test D (Noise - Optional Bonus):** Add Gaussian Noise ($\sigma = 0.1$) to the Test images and measure the accuracy drop.

Required Comparison Table:

Scenario	Overall Accuracy	Accuracy Drop	Biggest Drop (Class)	Most Confused Pair
A — Baseline	—	—	—	—
B — Grayscale	—	—	—	—
C — Channel Swap	—	—	—	—
D — Noise (Optional)	—	—	—	—

Analytical Question (Required): Which scenario caused the greatest accuracy drop? Did classes with low color distance in the EDA show the highest drop in the Grayscale test? What design implications does this finding have for machine vision systems in low-light environments or monochrome cameras?

6- Explainability with Grad-CAM

Implement Grad-CAM (using libraries like `pytorch-grad-cam`) on your model's final convolutional layer to visualize attention maps. Select at least two correct and two incorrect predictions from different classes, overlaying the generated heatmap on the original images. For a cross-scenario comparison, display the heatmap of a single image side-by-side for Scenario A (Color) and Scenario B (Grayscale). Have the model's attention regions shifted?

Analytical Question: In the incorrect predictions, does the heatmap indicate that the model focused heavily on the image background? If so, what techniques (e.g., Cutout Augmentation, Background Removal) could mitigate this spurious correlation? Note: All plots in this project must include explicit titles, axis labels, and legends to be graded.

Evaluation Criteria

Criterion	Weight	Description
Mathematical Calculation Accuracy (Section 3)	20%	Formula accuracy, Receptive Field, Trainable Params
EDA Quality and Plots	25%	Depth of color analysis, Distance Matrix, visual interpretation
Model Performance (Target: > 75%)	20%	Test Accuracy, Training Curves
Depth of Robustness Analysis	25%	Comparison table, analytical answers, connection to EDA
Grad-CAM Interpretation	10%	Heatmap quality, Background Bias analysis

Final Note

The goal of this assignment is not merely to achieve a high Accuracy score. The depth of your analysis and quality of your interpretation of the results is the most important evaluation criterion. A model with 80% accuracy but weak Robustness and Grad-CAM analysis is less valuable than a model with 75% accuracy that has thoroughly interpreted all aspects of the model's behavior.

Task 3 : RNN

Introduction

Knowledge Tracing (KT) is an important task in educational data mining and intelligent tutoring systems. The main goal of Knowledge Tracing is to model how students learn over time and to predict whether a student will answer the next question correctly.

In a Knowledge Tracing system, every student interaction is treated as part of a learning sequence. By analyzing previous answers, problem-solving behavior, and learning patterns, the model attempts to estimate the student's current knowledge state. These predictions can later be used in adaptive learning systems, personalized tutoring, recommendation systems, and student performance analysis.

Formally, a student interaction sequence in Knowledge Tracing can be represented as:

$$\{(q_1, m_1, r_1), (q_2, m_2, r_2), \dots, (q_T, m_T, r_T)\}$$

where:

- q_t represents the question, skill, or educational concept at timestep (t),
- m_t represents the metadata or behavioral information associated with that interaction, such as problem type, time spent, number of hints, or attempts,
- ($r_t \in \{0,1\}$) represents the student response outcome, where 0 indicates an incorrect answer and 1 indicates a correct answer.

In this project, students will build a deep learning model based on Recurrent Neural Networks to analyze sequential learning behavior from the ASSISTments2017 educational dataset.

1- Dataset Description

The project uses a processed subset of the ASSISTments2017 dataset. The dataset contains student interaction logs collected from an online learning platform. Each row represents one interaction between a student and an educational problem.

The dataset contains sequential student interaction records together with behavioral and performance-related features such as the exercised skill, correctness of responses, problem type, time spent solving questions, number of hints and attempts, historical error statistics, and timestamp.

Since Knowledge Tracing depends heavily on temporal learning behavior, the interactions are organized chronologically for each student using timestamps.

2- Preprocessing and Feature Engineering

Critical Requirement: You must carefully shuffle the train and test datasets at the sequence level to prevent biased training, while maintaining the chronological order of interactions within each student's sequence.

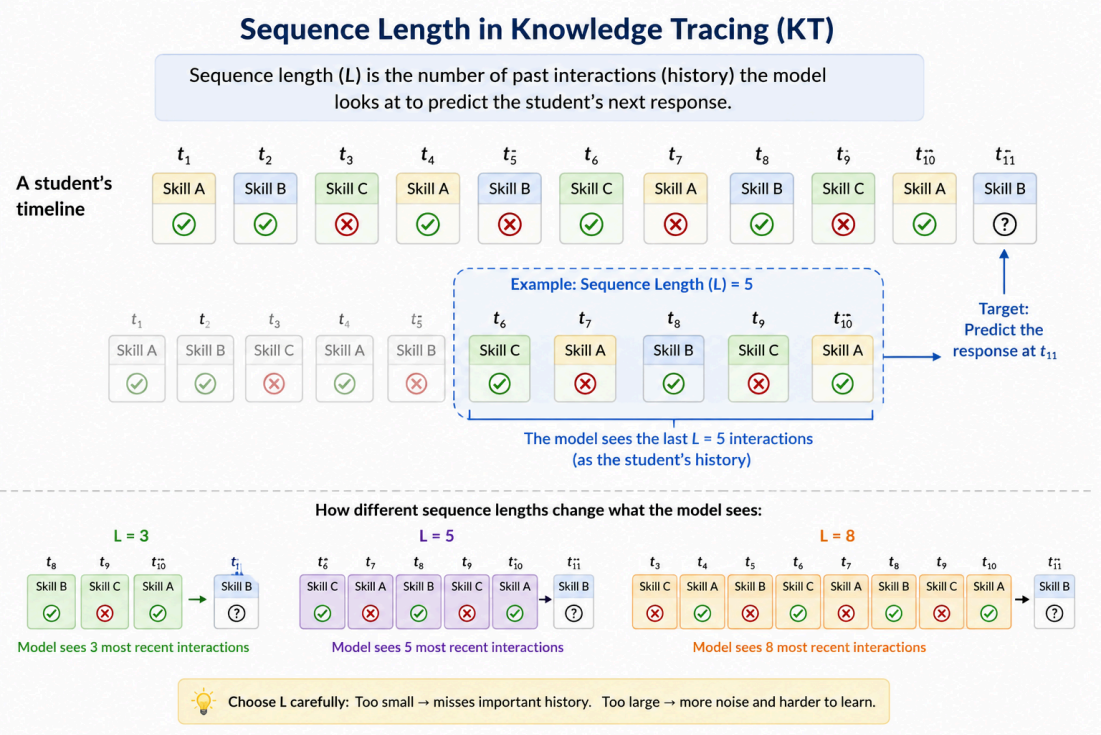
Raw educational datasets cannot be fed directly into deep learning models; therefore, designing a robust preprocessing pipeline is essential. You are expected to explore strategies that improve both model performance and training stability. Key preprocessing steps include:

Encoding Categorical Features: Deep learning models cannot directly process text values like skill names. You must convert categorical variables into numerical representations using techniques such as label encoding, index mapping, or embedding representations. Ensure that a specific index (usually 0) is reserved for sequence padding.

Sequential Representation: Knowledge Tracing is a sequential prediction problem. Instead of treating interactions independently, the model learns from previous student activity.

Student histories are transformed into fixed-length sequences using techniques such as sliding windows. For each sequence, the model uses previous interactions to predict the correctness of the next interaction.

This sequential structure is one of the most important aspects of the project.



Sequence length determines how many past student interactions the model can observe before predicting the next response. In Knowledge Tracing, the model learns from a student's recent learning history, such as previous skills, answers, and behaviors. A small sequence length may ignore important historical information, while a very large sequence length may introduce unnecessary noise and increase training complexity. Therefore, selecting an appropriate sequence length is important for balancing learning performance and computational efficiency.

Sequence Padding: Because students have varying numbers of interactions, you must apply padding techniques to ensure uniform sequence lengths within each training batch, adding special padding values (typically at the beginning of the sequence) to maintain consistent tensor dimensions.

Feature Scaling and Normalization: Numerical features (e.g., time spent, hint counts) exhibit drastically different ranges. Apply normalization techniques such as Standardization (Z-score), Min-Max scaling, or Logarithmic transformations for highly skewed features to improve convergence stability.

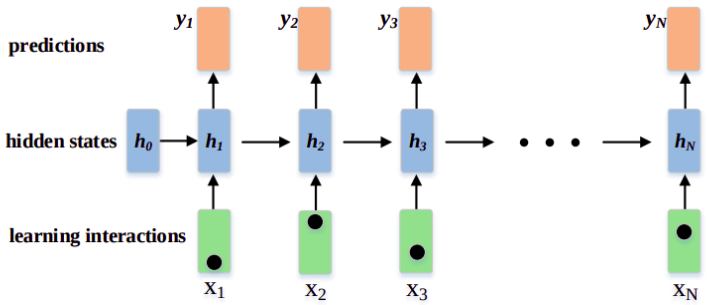
3- Exploratory Data Analysis (EDA)

Before training, conduct an exploratory analysis on the ASSISTments2017 dataset to understand correctness patterns, skill difficulties, and student ability relationships.

- **Visualize Correctness:** Plot correctness rates grouped by `skill` and `problemType` to identify the most challenging concepts.
- **Analyze Student Ability:** Examine the relationship between overall student ability (mean correctness) and help-seeking behaviors like timeTaken, hintCount, and attemptCount.
- **Correlation Analysis:** Calculate and plot correlations between features like frPast5WrongCount, consecutiveErrorsInRow, hintCount, and the target correct to understand how past struggles predict future performance.
- **Temporal Dynamics:** Plot learning curves showing correctness across interaction deciles to observe if students improve over time. Compare the first 10 versus the last 10 interactions to justify your sequence truncation/windowing strategy. (Attention: You must explicitly analyze and discuss the insights derived from each plot in your report).

4- Model Architecture:

Your model is a RNN that maps an input sequence of vectors $x_1, ..., x_T$ to an output sequence of vectors $y_1, ..., y_T$. This is achieved by computing a sequence of hidden states $h_1, ..., h_T$, which encodes relevant information from past observations to predict future student performance.



At each timestep t, the hidden state and prediction are computed as:

$$\mathbf{h}_t = \tanh\left(\mathbf{W}_{hx}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h\right),$$
$$\mathbf{y}_t = \sigma\left(\mathbf{W}_{yh}\mathbf{h}_t + \mathbf{b}_y\right),$$

where tanh is the non-linearity applied to the hidden state, σ(·) is the sigmoid activation function for binary classification, and the model is parameterized by the input weights (W_{hx}), recurrent weights (W_{hh}), readout weights (W_{yh}), and their respective biases (b_h, b_y).

Design an RNN utilizing PyTorch's recurrent layers to learn these temporal dependencies. Following the recurrent component, append one or more fully connected layers to generate the final binary prediction (0 for incorrect, 1 for correct). You are highly encouraged to apply regularization techniques, such as Dropout, to reduce overfitting.

5- Model Training

Train your RNN-based KT model using your preprocessed sequential data. Select an appropriate binary classification loss function (e.g., Binary Cross-Entropy) and an optimization algorithm (e.g., Adam).

- **Monitoring:** During training, continuously track the Training Loss, Validation Loss, Validation Accuracy, and Validation AUC score. Monitoring these metrics across epochs is crucial for identifying overfitting.
- **Hyperparameter Tuning:** Experiment with different configurations and document their impact on performance and computational cost. Key hyperparameters include sequence length, number of recurrent layers, hidden layer dimensionality, embedding dimensions, batch size, learning rate, and dropout rate.

6- Evaluation and Visualization

Evaluate your model's predictive performance on the test set using both numerical metrics and visual analysis.

Classification Metrics: Report standard binary classification metrics including Accuracy, Precision, Recall, and AUC. Briefly explain what each metric represents in the context of Knowledge Tracing and discuss which metric is the most informative for this specific task.

Confusion Matrix: Visualize the confusion matrix to analyze True Positives, False Positives, True Negatives, and False Negatives, identifying the specific types of errors your model makes.

Training Curves: Plot training and validation loss (as well as accuracy/AUC) across epochs to visualize convergence behavior and verify the absence of overfitting.

Prediction Analysis: Manually inspect and analyze a few examples of correct and incorrect predictions. Discuss what these instances reveal about the model's limitations and the complexities of student learning behavior.

7- Implementing an LSTM-Based Model (10% Bonus)

As an optional extension, replace the standard RNN layers with Long Short-Term Memory (LSTM) layers. LSTMs are advanced recurrent architectures designed to better capture long-term dependencies and mitigate the vanishing gradient problem inherent in vanilla RNNs. Redesign the recurrent component using one or more LSTM layers, experimenting with architectural choices like hidden dimensions and stacked layers. Train this LSTM model using the exact same preprocessing pipeline. Finally, evaluate and quantitatively compare both models. Your comparison should go beyond simply noting which model achieved higher accuracy; discuss *how* and *why* the different recurrent architectures behaved differently when modeling temporal educational patterns.

Questions (10% Bonus)

Question 1: Under what conditions is a MLP mathematically equivalent to Logistic Regression?

Question 2: Suppose you have a CNN that begins by taking an input image of size $28 \times 28 \times 3$ and passing it through a convolution layer that convolves the image using 3 filters of dimensions $2 \times 2 \times 3$ with valid padding.

- How many learnable parameters does this convolution layer have?
- Suppose that you instead decided to use a fully connected layer to replicate the behavior of this convolutional layer. How many parameters would that fully connected layer require?

Question 3: Explain mathematically the vanishing gradient problem in RNNs. Then, analyze how changing the sequence length (lookback window size) impacts the severity of this phenomenon.

Notes

- Upload your work as a zip file on the course website using the following format: **`DS_CA4_[Student_Number].zip`.**
- If the project is completed as a group, include the student numbers of all group members in the filename, but **only one member** should submit the final file.
- We will run your code during the project delivery session, so please ensure your results are fully reproducible.